Campus to Corporate: Tech Grooming Blueprint for Indian Engineering Graduates

Version 4 draft. Now organised by year, from first-year basics to job-ready skills. Status: early draft, seeking contributors and reviewers.

This open-source guide helps new engineering graduates in India move from the classroom to their first job. It does not replace your degree. It shows what your syllabus often leaves out: how real software, hardware and power systems are built, run and paid for.

Who it is for: students in all four years, from first-year to fresh graduates, in CSE, AI/ML, ECE, EE, Quantum and Math & Computing programmes.

How it works: every module is written by students and working engineers, and every module is reviewed by someone with real industry experience in that area.

How to use this guide

Start at your current year and read forward. Later years assume the earlier ones, so a second-year student should skim the Year 1 sections first.

Year	Focus	Read	By the end of the year you can
1	Understand what engineering and software are	Mindset, How an app works, Core concepts, Year 1 habits, Communication and soft skills	Explain how an app works from tap to database, and have finished one small project
2	Build fundamentals with real tools	Core concepts, Technical grinding (Git, APIs, data formats)	Use Git, call an API, and store and read data in a database
3	Specialise and gain experience	Branch guide, The AI impact, Careers in EV, renewable energy and chip design, Technical grinding, Industry tours	Show two projects on GitHub and have a plan for an internship
4	Become job-ready	Branch guide, edge example, Year-by-year checklist	Explain the design choices in your projects during an interview

Every module also carries a level (Year 1 to Year 4) and a status (Draft or Reviewed), described under CONTRIBUTING.md.

Repository layout

```
.github/
  ISSUE_TEMPLATE/            Forms for new modules, corrections and interview
stories
00-start-here/               Year 1
  what-is-engineering.md
  how-an-app-works.md
  year-1-habits.md
communication/               Years 1 to 4
  communication-and-soft-skills.md
01-core-concepts/            Years 1 to 2
  architecture-and-system-design.md
  layers-of-software.md      UI, front end, API, back end, middleware,
database
  data-roles.md              Analyst, engineer, scientist, ML engineer
02-branch-guide/             Years 1 to 4
  branch-guide.md            All six branches: maths base, syllabus gaps,
what to build
03-the-ai-impact/
  ai-advancements.md
  modern-developer-workflow.md
04-growth-careers/           Years 3 to 4
  ev.md
  renewable-energy.md
  chip-design.md
05-technical-grinding/       Years 2 to 4
  application-logic.md       APIs, Git, data formats
  edge-and-cloud-core.md     Running code close to the user
  electronics-to-software/
    secure-edge-ingest.ts    Worked example: device data into the cloud
06-industry-tours/           Years 3 to 4
  hyderabad.md
  bengaluru.md
  chennai.md
checklists/
  year-by-year.md
RESOURCES.md                 Free links, by branch and year
```

```
GLOSSARY.md
CONTRIBUTING.md
LICENSE                         Apache License 2.0
README.md                       Start here
```

Mindset: what is engineering?

Engineering is solving open-ended practical problems within limits: time, memory, network delay, power, safety and money. A solution that works only on a perfect lab setup is not finished.

Three shifts to make:

- **From remembering to deciding.** Exams reward recall. Jobs reward choosing between options and explaining why. Ask what each choice costs in CPU time, delay, power or rupees.
- **From the happy path to failure.** Labs are set up so everything works. Real systems face dropped networks, bad input and dead sensors. Build for what goes wrong first.
- **From single programs to connected systems.** Most work today means joining services, devices and networks securely. Learn how the pieces talk to each other.

Year 1: how an app works, from tap to screen

Every app you use is a chain of small parts that each do one job. Learn the chain once and most of software makes sense.

Say you tap **Order** in a food-delivery app. This happens in about a second:

1. **Front end:** The screen and buttons you see, running on your phone or in your browser. It collects your tap.
2. **API:** The app sends a request to a server in an agreed format, like a waiter carrying an order slip to the kitchen.
3. **Back end:** A program on a server applies the rules: is the item available, is the payment valid, which restaurant gets it?
4. **Database:** The back end saves the order and reads menus, prices and customer details from organised storage.
5. **Response:** The answer travels back the same way and the screen updates to say "Order placed".

Part	Restaurant	In the food app
Front end	Dining room and menu	Screens, buttons, forms

Part	Restaurant	In the food app
API	Waiter and order slip	The request from the app to the server
Back end	Kitchen	Order rules, payment checks
Database	Storeroom and record book	Menus, orders, customer records

If you are in ECE or EE, the same chain exists in hardware. In a smart electricity meter, a sensor measures power, a microcontroller processes it, a network module sends it, and a utility's server stores it and prepares the bill.

Try it today: Open any website in a desktop browser, press F12, and open the Network tab. Reload the page and watch the requests go out. Each line is one trip along the chain above.

Core concepts (Years 1 to 2)

These terms come up in almost every tech job, and most can be explained with the restaurant or the smart meter. Companies use some of them loosely, so this section gives the most common meaning and flags where it varies.

Architecture and system design

Architecture is the high-level plan of a system: its main parts, what each does and how they connect. Think of the floor plan of a building before any bricks are laid.

System design is the reasoning that turns that plan into something that works under real conditions: how many users, how fast, how much it may cost, and what happens when a part fails. For the restaurant, it is deciding that a Saturday rush needs two kitchens and a bigger storeroom.

In short, architecture is the picture and system design is the thinking behind it. Interviewers often ask you to design a system. There is no single right answer, because they want to hear your trade-offs.

Go deeper later: the System Design Primer is a free GitHub study guide on designing large-scale systems and preparing for system design interviews. It suits Years 3 and 4.

Branch	What architecture looks like
CSE / AI-ML	Boxes for the app, servers, database and services, with arrows showing which calls which

Branch	What architecture looks like
ECE	A block diagram of a board: processor, memory, sensors and radio
EE	A single-line diagram of a power network: generators, transformers, lines and loads
Quantum	Layers from qubits and control electronics up to the software that programs them
Math & Computing	The pipeline of a model or cryptographic system: data in, method, checks, result out

The layers of software

Term	What it is	Restaurant	Examples
UI (user interface)	Everything a person sees and touches: layout, colours, buttons, text	The menu design and the table setting	Wireframes and designs in Figma
Front end	The code that builds the UI and runs in the browser or phone	The dining room staff working from the menu	HTML, CSS, JavaScript, React, Android, iOS
API	An agreed way for one program to ask another for something	The order slip	REST and JSON over HTTP
Back end	Server code that applies the rules and talks to the database	The kitchen	Python, Java, Node.js, Go
Middleware	Software between other parts that handles shared jobs, such as checking who you are, logging, limiting requests or passing messages	The host who checks your booking before seating you	Login checks, message queues
Database	Organised, lasting storage for data	The storeroom and record book	PostgreSQL, MySQL (tables); MongoDB (flexible documents)

Two things beginners often mix up: UI is the design of what people see, while front end is the code that makes it work, though people use both words for the same thing. "Middleware" varies most between companies. Some use it for message queues, others for any layer between the front end and the back end.

If you are unsure where to start with databases, learn SQL first. It is used almost everywhere and the ideas carry over to other tools.

Data roles compared

The water-supply picture helps: a **data engineer** builds the pipes and treatment plant, a **data analyst** reads the meters and reports usage, and a **data scientist** studies the patterns to predict demand.

Role	Main question	Typical work	Core skills
Data analyst	What happened?	Cleans data, builds reports and dashboards	SQL, Excel, a dashboard tool
Data engineer	How does the data arrive reliably?	Builds pipelines that collect, clean and store data at scale	SQL, Python, cloud storage, pipeline tools
Data scientist	Why did it happen, and what will happen next?	Runs experiments, builds statistical and learning models	Statistics, Python, machine learning
ML engineer	How do we run a model reliably in a product?	Deploys, monitors and updates models	Python, software engineering, MLOps

Data engineering sits closer to software engineering, and data science sits closer to statistics. Small companies often combine roles, so always read the job description rather than the title.

Year 1 habits

Three habits formed in first year pay off for all four: study by building, use AI as a tutor rather than a shortcut, and finish one small project.

Study by building

- After each topic, write down one small thing you could make or test with it, then try it.
- Keep notes in your own words, and redo a solved problem later without looking.

- Use free, reliable sources: your college library, NPTEL courses and the official documentation of the tools you use.

Go deeper: NPTEL offers free recorded courses from IIT and IISc faculty through SWAYAM, across engineering fields. Learning is free; a certificate needs a paid exam.

Use AI as a tutor, not a shortcut

- Ask for an explanation in a different way, then explain it back in your own words.
- Ask for a hint before the answer.
- Run every piece of code it gives you and read every line. If you cannot explain a line, ask about it.
- AI can be confidently wrong. Check important facts against your textbook or the official documentation.
- Never submit work you cannot explain, and follow your college's rules on AI use in assignments.

Your first project (two to four weeks)

Pick one that fits your branch:

Branch	First project
CSE / AI-ML	A to-do list web app: a simple page, a small server and a database that saves your tasks
ECE / EE	Read a temperature or light sensor with an Arduino or ESP32 and show the value on screen
Quantum / Math & Computing	Implement a small algorithm, such as a simple cipher, and plot how its run time grows with input size

To count as finished, the project should be small enough to complete, live on GitHub, and have a README that says what it does and how to run it. Add one paragraph on something that went wrong and how you fixed it. That paragraph is what interviewers enjoy reading.

Communication and soft skills (Years 1 to 4)

Communication is a skill you build with regular practice, not with one course, and it matters in interviews, group projects and daily work. Start in Year 1 and keep going.

Year	Practise	Free resources
1	Explain one concept a week to a friend in two minutes, without jargon. Build comfort with spoken and written English.	British Council LearnEnglish (free materials); NPTEL's Developing Soft Skills and Personality from IIT Kanpur
2	Write a clear README for every project. Present a project in five minutes.	Google's free Technical Writing courses, aimed at software engineers and CS students
3	Write professional emails for internships. Build a clear LinkedIn profile. Give and receive feedback on group projects.	NPTEL's Soft Skill Development from IIT Kharagpur, which covers communication, interpersonal skills and presentation
4	Do mock interviews. Practise explaining your design choices out loud.	NPTEL's soft skills training, offered free to learners who score well in an NPTEL course

NPTEL course content is free to learn from. A certificate needs a paid exam, and fees and eligibility can change, so check the course page.

Habits that help in every year

- **Short and clear beats long and fancy.** Use plain, short sentences. Accuracy matters more than impressive vocabulary.
- **Read documentation daily.** It builds technical English and shows how engineers write.
- **Join a speaking group.** A college debate or tech-talk club, or a local club such as Toastmasters, gives you regular practice and feedback.
- **Ask for feedback and act on it.** After a talk or a code review, write down one thing you will change.
- **Practise in the open.** Post a short project write-up, answer a question in a community, or explain a concept in a short video.
- **If English is your second language,** that is common among engineers worldwide. Aim to be understood, and practise speaking aloud, not only reading and writing.

Branch guide (Years 1 to 4)

Each row gives the maths you will lean on, how AI is changing the field, and what the industry actually produces.

Branch	Maths you will use	How AI is changing it	What the industry builds
CSE	Discrete maths, logic, complexity	AI tools write routine code, so design, testing and security matter more	Web platforms, databases, cloud services
AI/ML	Linear algebra, probability, calculus	Less building models from scratch, more adapting them, running them reliably and controlling cost	Recommendation systems, language-model applications, automated software agents
ECE	Signals, electromagnetics, digital logic	Strong demand for chips and boards that run AI efficiently	Connected devices, sensors, chips, communication modules
EE	Differential equations, complex analysis, circuit theory	Data centres need large, reliable power and cooling	Smart grids, industrial machines, power systems
Quantum	Linear operators, quantum mechanics, solid-state physics	Early-stage field; AI helps with design and error handling	Experimental quantum hardware, secure communication research
Math & Computing	Number theory, abstract algebra, statistics	Mathematical foundations for new model designs	Quantitative finance, cryptography, core AI research

Where the syllabus stops, and what to practise instead

These are common patterns, not rules. Syllabi differ between universities, and many colleges already teach some of this. Use the list as a self-check.

Branch	Common syllabus gap	Practise this instead
CSE	Writing code on paper, rarely running it in a real environment	Package an app with Docker, add logging and basic monitoring (for example Prometheus), and read logs to find a fault

Branch	Common syllabus gap	Practise this instead
AI/ML	Doing gradient calculations by hand, with little on deployment	Build a small retrieval-based question-answering app, index documents in a vector database (pgvector or Chroma), and track what each request costs
ECE	Memorising 8085/8086 pin layouts	Program a modern microcontroller such as the ESP32, send readings over Wi-Fi, and handle JSON payloads and lost connections
EE	Solving idealised circuits with clean numbers	Log real power data from a small load, study how noise and load changes affect it, and read a basic grid or inverter datasheet
Quantum	Heavy theory with no hands-on contact	Run circuits on free cloud simulators (for example Qiskit) and compare noisy and ideal results
Math & Computing	Proofs with no computation	Implement an algorithm you studied, such as RSA or a basic statistical model, test it on real data and explain where it breaks

The fundamentals still matter. The point is to pair each one with something you have actually built and run.

Go deeper: roadmap.sh has community-made roadmaps by role, and workat.tech has a roadmap for first and second-year students. Both lean towards software roles, so use the tables above for other branches.

The AI impact

AI tools change what you spend your hours on. They do not remove the need to understand what you are building.

- **Software:** Routine code is faster to produce. Reviewing it, testing it, securing it and judging design trade-offs become the main skills.
- **Electronics:** Demand grows for efficient chips, sensors and low-power devices that run models close to where data is collected.
- **Power:** Large computing sites need steady power and cooling. This opens work in grid planning, storage and energy efficiency.

- **Research:** Strong maths lets you understand why a model behaves as it does, not only how to call it.

Modern developer workflow

1. Define the problem and its limits (speed, cost, safety) before writing anything.
2. Use AI tools for drafts, boilerplate and explanations.
3. Read every line you accept. You are responsible for it.
4. Write tests, run them and check failure cases.
5. Deploy small, monitor and fix.

Careers in EV, renewable energy and chip design (Years 3 to 4)

These three growing fields hire across branches, but the routes differ. The table suggests entry points for each branch. Treat it as a starting guide, not a promise of jobs, because roles vary by company.

Branch	Electric vehicles (EV)	Renewable energy	Chip design
CSE	Vehicle software, telemetry and cloud platforms	Monitoring and control software for solar, wind and storage	Design tools, scripting, verification software and firmware
AI/ML	Battery health and range prediction, predictive maintenance	Forecasting power generation and demand, fault detection	Analysing design and test data (an emerging area)
ECE	Battery management electronics, embedded control, sensors, vehicle networks	Inverter and sensor electronics, IoT monitoring	Core digital and analog design, verification
EE	Power electronics, motors, charging systems	Power systems, grid connection, inverters, storage	Power-management and analog circuits
Quantum	Mostly research, such as battery materials modelling, usually after postgraduate study	Research on materials such as solar cells, usually postgraduate	Semiconductor device physics and quantum hardware research

Branch	Electric vehicles (EV)	Renewable energy	Chip design
Math & Computing	Simulation, optimisation and control algorithms	Forecasting and optimisation models	Algorithms for design automation, security and cryptography hardware

Job listings in these fields often name electrical, electronics and mechanical degrees. If you are in CSE, AI/ML, Quantum or Math & Computing, your route usually runs through software, data or research, so project proof matters even more. Mechanical and civil routes will be added when those branches are.

Electric vehicles

- EV jobs are not only for automobile engineers. One Naukri campus guide says electrical, electronics, computer science and chemistry graduates also find roles.
- Skills often asked for include battery systems and battery management, power electronics, thermal management, embedded software and vehicle networks such as CAN. See this skills-gap overview and this role list.
- NPTEL has added electric vehicle courses. Search NPTEL for the current list before paying for a private course.
- **Project:** extend the edge example into a battery monitor. Use an ESP32 to read the voltage and temperature of a small low-voltage cell, send the signed readings to the edge service, and chart them. Never work on high-voltage battery packs without qualified supervision.

Renewable energy

- Government programmes sit with MNRE: Suryamitra for solar technicians, Vayumitra for wind, and a National Renewable Energy Internship Programme. See the MNRE training and internship page.
- A reality check: Suryamitra is technician-level, and IndiaSpend reporting found closed centres and weak job outcomes in some areas. Engineers should aim for internships and projects first.
- The sector is getting more digital, with monitoring, battery storage, green hydrogen, AI and IoT, according to the CEO of the Skill Council for Green Jobs in Renewable Watch. That opens doors for CSE, AI/ML and ECE students.
- **Project:** build a small solar monitoring dashboard with a panel, a sensor and the edge example, or forecast generation from public weather data.

Chip design

- MeitY's Chips to Start-up (C2S) programme, launched in 2022 with C-DAC as the nodal agency, gives students hands-on chip design, fabrication and testing support with professional design tools. MeitY reports over 68,000 students trained.
- C2S works through enrolled colleges, so ask your ECE or EE department whether your college takes part.
- Chip design and chip manufacturing need different skills. C2S focuses on design.
- Search NPTEL for digital electronics and VLSI courses, and practise by simulating small circuits in a free simulator.
- **Project:** design and simulate a small digital circuit, such as a counter or a traffic-light controller, and document its waveforms.

Technical grinding (Years 2 to 4)

Application logic

Three basics every employer expects:

- **APIs:** How one program asks another for data or action. Practise calling a public API, handling errors and respecting rate limits.
- **Git:** Branches, commits, pull requests and resolving conflicts. Use it for every project, including solo ones.
- **Data formats:** Read and write JSON and CSV. Validate input before you trust it.

Go deeper: MIT's Missing Semester of Your CS Education is a free course on practical tools that syllabi often skip, such as the command line and Git.

Edge and cloud core

An *edge* service runs close to the user or device instead of in one distant data centre. This cuts delay and lets you reject bad traffic early. Platforms such as Cloudflare Workers are a common way to try it.

Worked example (Year 3 and up): receiving data from an ESP32 sensor

A small device sends a reading every few seconds. The edge service must confirm the message is genuine, reject old or malformed ones, convert the raw value, and pass it on without making the device wait.

Design choices in this example:

- Each device has its own secret, so one stolen key does not expose the whole fleet.

- Messages are signed with HMAC-SHA256 and rejected if the timestamp is more than five minutes off. This limits replay of captured messages. Fully stopping replays also needs a counter or one-time value.
- Bad JSON returns 400, bad signature returns 401, and messages are plain and honest.
- The ESP32 must set its clock, for example through NTP, or every timestamp will fail.
- The conversion assumes a 12-bit reading (0 to 4095) with a 3.3 V reference. The ESP32's built-in ADC is not perfectly linear, so calibrate for precise work.

```
// Cloudflare Worker: secure telemetry ingest (teaching example, not a
finished product)

interface Env {
  DEVICE_SECRETS: string;   // JSON map, e.g. {"esp32-001": "long-random-
secret"}
  ORIGIN_API_URL: string;
  ORIGIN_API_TOKEN: string;
}

interface Telemetry {
  deviceId: string;
  timestamp: number;        // milliseconds since 1970
  rawAdc: number;           // 12-bit reading, 0 to 4095
  chipTemperature: number;
  signature: string;        // hex HMAC-SHA256 of the message below
}

const MAX_CLOCK_SKEW_MS = 5 * 60 * 1000;

const json = (body: unknown, status: number): Response =>
  new Response(JSON.stringify(body), {
    status,
    headers: { "Content-Type": "application/json" },
  });

function isTelemetry(x: unknown): x is Telemetry {
  if (typeof x !== "object" || x === null) return false;
  const t = x as Record<string, unknown>;
  return (
    typeof t.deviceId === "string" &&
    typeof t.timestamp === "number" &&
    typeof t.rawAdc === "number" && t.rawAdc >= 0 && t.rawAdc <= 4095 &&
    typeof t.chipTemperature === "number" &&
    typeof t.signature === "string"
  );
}
```

```
function hexToBytes(hex: string): Uint8Array {
  const out = new Uint8Array(hex.length / 2);
  for (let i = 0; i < out.length; i++) {
    out[i] = parseInt(hex.slice(i * 2, i * 2 + 2), 16);
  }
  return out;
}

async function verifySignature(p: Telemetry, secret: string):
Promise<boolean> {
  if (!/^[0-9a-f]{64}$/i.test(p.signature)) return false;
  const encoder = new TextEncoder();
  const key = await crypto.subtle.importKey(
    "raw",
    encoder.encode(secret),
    { name: "HMAC", hash: "SHA-256" },
    false,
    ["verify"]
  );
  const message =
`${p.deviceId}.${p.timestamp}.${p.rawAdc}.${p.chipTemperature}`;
  // crypto.subtle.verify compares in constant time
  return crypto.subtle.verify("HMAC", key, hexToBytes(p.signature),
encoder.encode(message));
}

async function forwardToOrigin(env: Env, deviceId: string, volts: number):
Promise<void> {
  try {
    const res = await fetch(env.ORIGIN_API_URL, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${env.ORIGIN_API_TOKEN}`,
      },
      body: JSON.stringify({ device: deviceId, volts }),
    });
    if (!res.ok) console.error(`Origin returned ${res.status} for
${deviceId}`);
  } catch (err) {
    console.error("Forwarding to origin failed", err);
  }
}

export default {
```

```
  async fetch(request: Request, env: Env, ctx: ExecutionContext):
Promise<Response> {
    if (request.method !== "POST") return json({ error: "Method not allowed"
}, 405);

    let body: unknown;
    try {
      body = await request.json();
    } catch {
      return json({ error: "Body must be valid JSON" }, 400);
    }
    if (!isTelemetry(body)) return json({ error: "Missing or invalid fields"
}, 400);

    if (Math.abs(Date.now() - body.timestamp) > MAX_CLOCK_SKEW_MS) {
      return json({ error: "Timestamp outside allowed window" }, 401);
    }

    const secrets = JSON.parse(env.DEVICE_SECRETS) as Record<string, string>;
    const secret = secrets[body.deviceId];
    if (!secret || !(await verifySignature(body, secret))) {
      return json({ error: "Invalid signature" }, 401);
    }

    const volts = (body.rawAdc / 4095) * 3.3;

    // Reply now; forward in the background so the device can go back to
sleep
    ctx.waitUntil(forwardToOrigin(env, body.deviceId, volts));
    return json({ status: "accepted" }, 202);
  },
};
```

Exercises: (1) Write the matching ESP32 code that builds the same message and signs it. (2) Add a per-device counter to block replays. (3) Store readings and draw a chart.

Industry tours (Years 3 to 4)

These are self-guided, one city per tour. Do one tour in the city nearest you.

Before you go: most company campuses and startup hubs are not open to walk-ins. Check event pages and meetup listings, ask alumni or your placement cell for introductions, and book anything that needs it. Never enter restricted areas. Data centres are not visitable, so this guide does not pretend otherwise.

Every tour follows the same three days:

- **Day 1, the startup scene.** Attend a public event, demo day or meetup at a startup hub or co-working space. Note how teams plan work, review code and talk about users.
- **Day 2, how big companies work.** See the large tech parks from outside and meet people who work inside them (alumni, mentors, community events). Ask what a normal week looks like, how code reaches users, and what they wish they had learned in college. Add a science or technology museum if one is nearby.
- **Day 3, build the connection.** Use a maker space, lab or your own laptop to run the edge example above with a real or simulated sensor. Seeing data travel from device to cloud makes the diagrams real.

City	Day 1 starting points	Day 2 areas to explore	Day 3 option
Hyderabad	T-Hub and T-Works events	HITEC City and Gachibowli tech areas	Maker space or university lab
Bengaluru	Startup and developer community meetups	Manyata Tech Park area and other large parks	Maker space or co-working lab
Chennai	IITM Research Park events	OMR and Guindy tech corridors	Maker space or college lab

Verify each place's current access rules before you plan your trip. Details change often.

Year-by-year checklist

Tick these off as you go. Each year builds on the one before, and none needs a paid course.

Year 1

- [ ] Explain the path from a tap to the database in your own words
- [ ] Learn one programming language well enough to write small programs without help
- [ ] Create a GitHub account and learn the basics of Git: commit and push
- [ ] Finish one small project and put it on GitHub
- [ ] Join one college club or technical community

Year 2

- [ ] Learn SQL basics and use a database in a project
- [ ] Call a public API and handle errors properly
- [ ] Learn Git branches and pull requests

- [ ] Build one project with all the layers: front end, back end and database (or sensor, microcontroller and server)
- [ ] Read one technical article a week and note one new idea

Year 3

- [ ] Use the Branch guide to choose a direction, and talk to one person working in it
- [ ] Have two projects on GitHub with clear READMEs
- [ ] Prepare a one-page resume and apply for internships
- [ ] Attend a meetup or hackathon and complete one industry tour
- [ ] Contribute to an open-source project, even a documentation fix

Year 4

- [ ] Deploy a project that other people can open, such as the edge example
- [ ] Practise explaining one project's design: parts, trade-offs and what happens when something fails
- [ ] Prepare interview notes using the day-one checklist format
- [ ] Do mock interviews with a senior or an alumnus
- [ ] Ask a working engineer to review your projects

Free resources and community

Do not rewrite what already exists. Link to it, and spend your effort on what is missing. Most of these lean towards software, which is the gap this guide fills for other branches.

Resource	Best for	Branches	Years
NPTEL on SWAYAM	Free recorded courses from IIT and IISc faculty; a certificate needs a paid exam	All	1 to 4
The Missing Semester (MIT)	Practical tools such as the command line and Git	Software-heavy, useful to all	2
System Design Primer	Large-scale system design and interview preparation	CSE, AI/ML	3 to 4
roadmap.sh	Community roadmaps by role and skill	Software roles	2 to 4
workat.tech first and second-year roadmap	An early-years software roadmap	Software roles	1 to 2

Resource	Best for	Branches	Years
r/developersIndia community wiki	Community guides on topics such as hackathons, jobs and cybersecurity	Software, open to all	1 to 4
Chips to Start-up (C2S)	Government-backed chip design training through enrolled colleges	ECE, EE, CSE	2 to 4
MNRE training and internships	Renewable energy skill programmes and internships	EE, ECE, CSE	3 to 4
Google Technical Writing	Clear technical writing for engineers	All	2 to 4
British Council LearnEnglish	Free English practice materials	All	1 to 4
NPTEL soft skills courses	Communication, presentation and interpersonal skills	All	1 to 4

If you want to contribute to this guide, see CONTRIBUTING.md below.

CONTRIBUTING.md

Write for a fresher who is smart but new. Keep modules short, scannable and plain.

Module template

1. **The textbook definition.** The version you were taught.
2. **The production reality.** The same idea explained with a physical analogy or simple process. No jargon without a one-line explanation.
3. **The trade-off table.** Options compared against real limits such as cost, speed and risk.
4. **Day-one checklist.** Interview talking points and common mistakes in production.
5. **Reviewed by.** Name, role and date of the working engineer who checked it.

Tag every module with a **level** (Year 1, 2, 3 or 4) and a **status** (Draft or Reviewed). Year 1 modules stay under one page, explain every term on first use, and include an analogy for each branch they apply to.

Rules

- Every factual claim should be checkable. Link a source or say it comes from your experience.

- Code examples must run. State the versions you tested.
- Describe patterns as patterns, not absolutes. Syllabi and companies differ.
- Be kind in reviews. Contributors are learning too.

Link, don't copy. Before writing a module, check whether a good free resource already exists. If it does, link to it and add only what it leaves out. Recheck every link when a module is reviewed.

How to contribute: start here

You do not need to be an expert to help. Students can share experiences, working engineers can review, and anyone can fix a typo. These are starting rules, and the community can change them as it grows.

Ways to help

Who you are	What you can do	Time
First or second-year student	Point out a confusing paragraph, ask a question you wish was answered, or add a glossary term	15 minutes
Any student	Share a campus, internship or interview experience, without confidential details	1 hour
Final-year student or graduate	Write or improve a module using the template	A few hours
Working engineer	Review a module for accuracy and add your name as reviewer	30 to 60 minutes
Faculty or mentor	Check the branch tables and suggest corrections	30 minutes
Translator	Translate a Year 1 module into Hindi or a regional language	A few hours

Your first contribution, no coding needed

1. Create a free GitHub account.
2. Open the repository and pick an issue labelled "good first issue", or open a new issue using one of the templates.
3. Comment "I'll take this" so others know it is taken.
4. For a small edit, open the file on GitHub, click the pencil icon, make your change and choose "Propose changes". GitHub then opens a pull request for you.

5. A reviewer replies, aiming for within seven days. Answer their comments. When it is approved, a maintainer merges it and your name goes on the contributors list.

Roles

Role	What they do	How to become one
Contributor	Proposes changes	Open a pull request
Reviewer	Checks one area for accuracy and adds the Reviewed label with name, role and date	Offer to review, with a short note on your experience, after one accepted contribution
Maintainer	Merges changes, keeps the structure tidy and applies the community rules	Invited by current maintainers after steady contributions

Review checklist

- Is every factual claim correct, or linked to a source?
- Does the module follow the template, including Level, Status and Reviewed by?
- Is every term explained on first use, with an analogy for each branch it applies to?
- Do the code examples run, with versions stated?
- Do all links open, and does the module link to existing free resources instead of copying them?
- Is it fair to every branch and college, without stereotypes?

Community rules

- Be respectful. Beginners ask basic questions, and no one is mocked for them.
- Disagree with ideas, not people.
- Do not share personal data, confidential employer information or leaked exam and interview papers.
- Do not paste copyrighted course material. Link to it and summarise it in your own words.
- Do not add paid-course promotions or referral links. If you work for a company you mention, say so.
- Report concerns to the maintainers. Consider adopting an established code of conduct, such as the Contributor Covenant, instead of writing your own.

Licence and credit

The repository uses the Apache License 2.0. By contributing, you agree your work is shared under the same licence, so submit only work you wrote or have the right to share. Credit contributors with the all-contributors convention, which the r/developersIndia wiki repository also uses.

Good first issues

- Write a Hindi or regional-language summary of "How an app works".
- Add one more branch analogy to "Architecture and system design".
- Check every link in "Free resources and community" and report broken ones.
- Add an interview story using the module template.
- Write the Hyderabad, Bengaluru or Chennai tour page with verified access details.
- Review the ECE or EE rows of the branch tables (for reviewers).
- Write a Year 1 module on the basics of version control.
- Draft a civil or mechanical branch row, marked Draft.

Where to talk

Start with GitHub Discussions for questions and ideas, so everything stays in one searchable place. Add a chat group later if people ask for one.

Decisions for the founders

- [ ] Project name and repository address
- [ ] A contact email for conduct reports
- [ ] Which code of conduct to adopt
- [ ] Review response time to promise, such as seven days
- [ ] Who the first three reviewers and maintainers are
- [ ] Whether and when to open a chat channel

Glossary

Plain-language meanings of the terms used in this guide, in alphabetical order.

Term	Meaning
API	An agreed way for one program to ask another for data or action

Term	Meaning
Architecture	The high-level plan of a system: its parts and how they connect
Back end	Server code that applies the rules and talks to the database
Data pipeline	An automated route that collects, cleans and stores data so others can use it
Database	Organised, lasting storage for data
Edge	Running code close to the user or device instead of in one distant data centre
Front end	The code that builds what users see and runs in their browser or phone
Git	A tool that records every change to your files so you can go back and work with others
HMAC	A code created with a shared secret that proves a message is genuine and unchanged
JSON	A simple text format for sending structured data between programs
Middleware	Software between other parts that handles shared jobs such as login checks and logging (meaning varies by company)
MLOps	The practice of deploying, monitoring and updating machine-learning models reliably
Replay attack	Resending a captured genuine message to trick a system
SQL	The standard language for asking questions of a table-based database
System design	Choosing how to build a system so it meets real needs for speed, cost and reliability
UI	The user interface: what people see and touch

Launch posts

Publish only after at least three modules are complete and reviewed.

LinkedIn

Many Indian engineering graduates are strong on theory and unsure what day one at work looks like. I have started an open-source guide to help with that.

It is organised by year, so first-years can start with how apps work and what architecture, front end, back end and databases mean, while final-years finish with interview preparation. It covers six branches, what each syllabus often leaves out, and hands-on practice to fill the gap. It also has a worked example of sending sensor data securely from a chip to the cloud, plus city tour plans for Hyderabad, Bengaluru and Chennai.

This is an early draft. I am looking for students to add campus experiences and working engineers to review modules for accuracy. If that is you, the repository link is in the comments.

Reddit (r/developersIndia)

Title: Early draft of an open-source guide for fresh engineering graduates. Looking for feedback and reviewers

Hi all. I am building a free guide on the gap between college syllabi and real jobs, covering CSE, AI/ML, ECE, EE, Quantum and Math & Computing. So far it has year-by-year basics for first-years (how apps work, architecture, front end, back end, databases and data roles), a branch guide, a worked edge-computing example and city tour plans.

I would like to know what you wish you had learned before your first job, and I need working engineers to review modules. Link below. Please be honest about what is wrong or missing.

Check the subreddit's current rules on self-promotion and flair before posting.